INSTITUTE: DEDAN KIMATHI UNIVERSITY OF TECHNOLOGY

SCHOOL: COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

DEPARTMENT: COMPUTER SCIENCE

COURSE: BACHELOR OF SCIENCE IN COMPUTER SCIENCE

UNIT CODE: CCS 3105

UNIT NAME: SYSTEMS PROGRAMMING

TASK: TERM PAPER – WEEK 6 (MULTITHREADING AND THREAD SYNCHRONIZATION)

DATE OF SUBMISSION: 3rd August 2026

Submitted to: george.musumba@hotmail.com

NAME: JOSEPH NDIRANGU CHEGE

REGISTRATION NUMBER: C026-01-0807/2024

Question One (20 Marks)

Thread Creation and Management

A software company is developing a task scheduling application where multiple tasks must execute concurrently.

Develop a multithreaded C program using POSIX Threads that:

a) Creates five worker threads, each displaying:

- Its thread identifier.
- A unique task number.
- A message indicating that it has started execution. (6 Marks)

b) The main thread should wait for all worker threads to complete using pthread_join() before displaying the message:

All scheduled tasks have completed successfully.

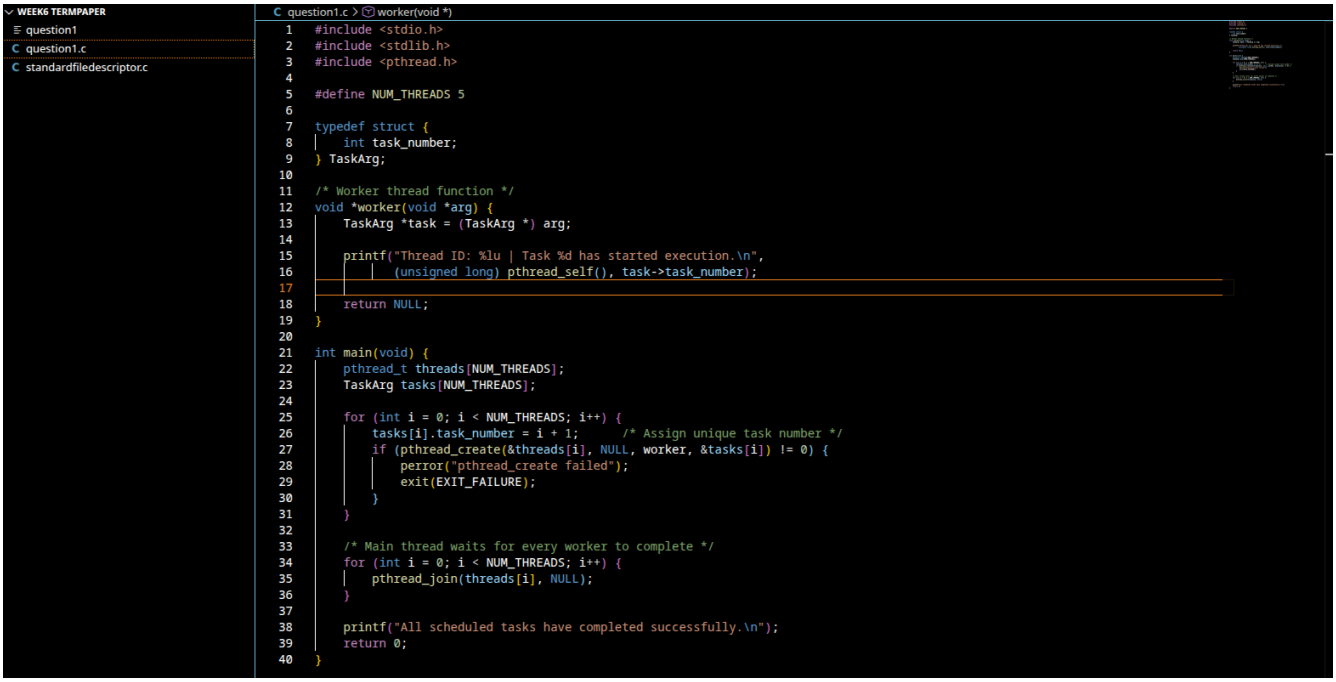
(4 Marks)

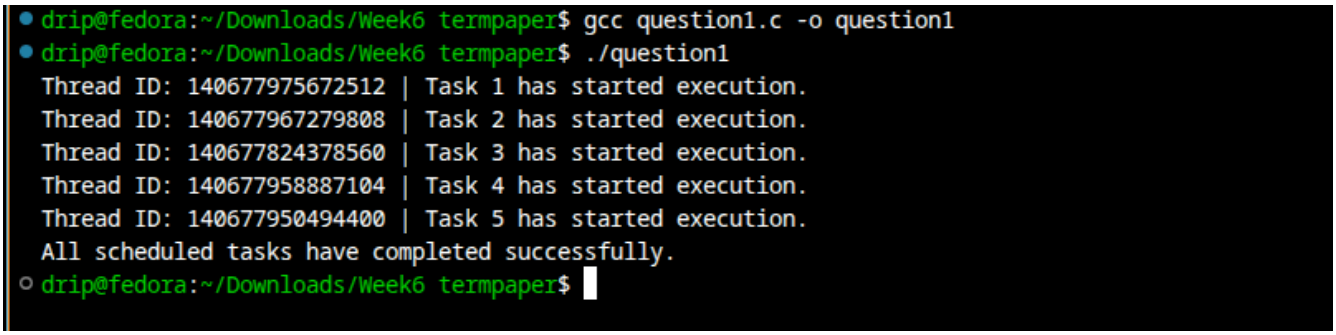
c) Modify the program so that each worker thread receives its task number as an argument instead of using a global variable. Explain the advantages of passing parameters to threads. (4 Marks)

- Correctness under Concurrency: When a global counter variable gets used, the global variable may be updated by the main thread before a new thread reads it. Passing a dedicated TaskArg structure per thread guarantees that each thread has its own private copy of the data, which will not be overwritten.
- No synchronization overhead: Because each thread only reads its own isolated parameter struct, no mutex locks are required to read task numbers.
- Reusability & Clarity: Parameterized thread functions can be called into multiple execution contexts without accessing global variables.

d) Include screenshots showing:

- Source code.
- Successful compilation.
- Program execution.
- Output demonstrating that all five threads executed successfully. (6 Marks)





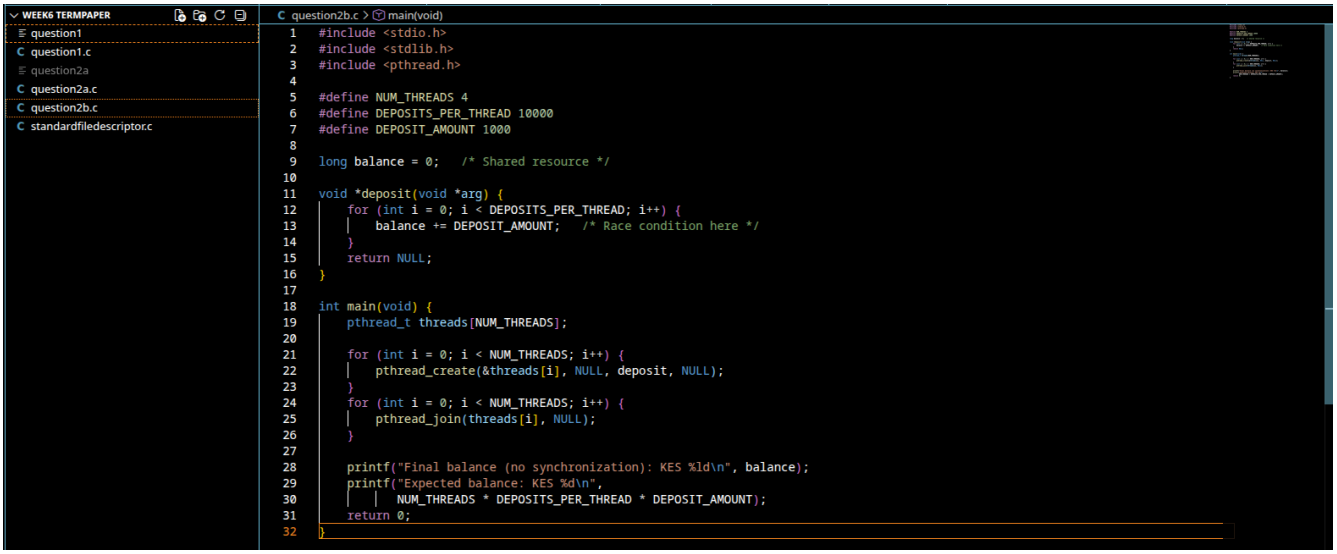
Question Two (20 Marks)

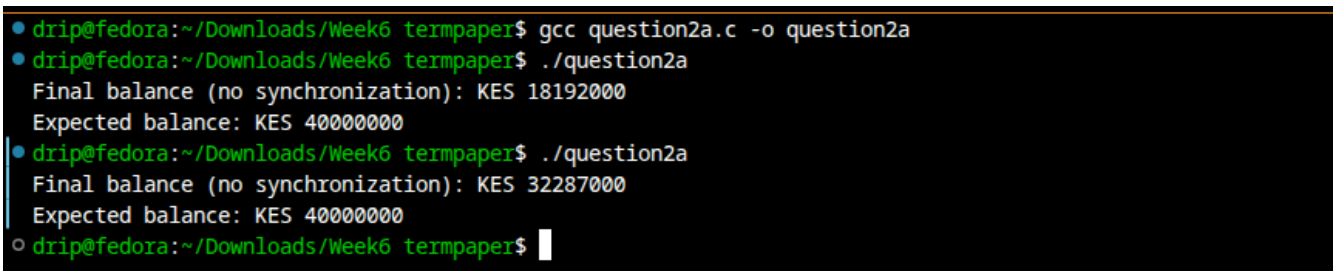
Race Conditions and Mutex Synchronization

A bank is developing an ATM system where multiple ATM terminals update the same customer account simultaneously.

Develop a multithreaded program that:

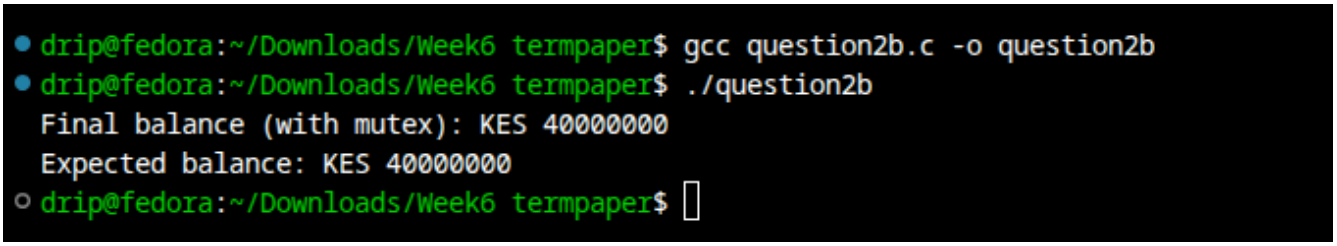
a) Creates four threads, each depositing KES 1,000 into a shared bank balance 10,000 times without synchronization. Display the final account balance and explain your observations. (6 Marks)





b) Modify the program by protecting the shared balance using a POSIX mutex (pthread_mutex_lock() and pthread_mutex_unlock()). (6 Marks)

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 4
#define DEPOSITS_PER_THREAD 10000
#define DEPOSIT_AMOUNT 1000

long balance = 0;
pthread_mutex_t balance_lock;

void *deposit(void *arg) {
    for (int i = 0; i < DEPOSITS_PER_THREAD; i++) {
        pthread_mutex_lock(&balance_lock);
        balance += DEPOSIT_AMOUNT;      /* Protected critical section */
        pthread_mutex_unlock(&balance_lock);
    }
    return NULL;
}

int main(void) {
    pthread_t threads[NUM_THREADS];
    pthread_mutex_init(&balance_lock, NULL);

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_create(&threads[i], NULL, deposit, NULL);
    }
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    pthread_mutex_destroy(&balance_lock);
    printf("Final balance (with mutex): KES %ld\n", balance);
    printf("Expected balance: KES %d\n",
        NUM_THREADS * DEPOSITS_PER_THREAD * DEPOSIT_AMOUNT);
    return 0;
}
```



c) Compare the outputs obtained before and after using the mutex, explaining the concept of race conditions and critical sections. (4 Marks)

Without synchronization, balance += DEPOSIT_AMOUNT compiles down to three separate steps: read the current value, add to it, write it back. A race condition occurs whenever two or more threads interleave those steps on the same shared variable — for example, both threads read the same old value before either writes back its increment, so one of the two deposits is silently lost. This is why the unsynchronized run produces a final balance lower than expected, and a different wrong value on each run, since the exact interleaving depends on unpredictable OS scheduling.

The pthread_mutex_lock()/pthread_mutex_unlock() pair around the increment turns balance += DEPOSIT_AMOUNT into a critical section — a region of code that only one thread may execute at a time. Any other thread that calls pthread_mutex_lock() while the section is occupied simply blocks until it is released, so every deposit is applied in full before the next one starts, and the final balance always matches the expected value.

d) Include screenshots showing:

- Compilation.
- Output before synchronization.
- Output after synchronization. (4 Marks)

Question Three (20 Marks)

Producer–Consumer Problem

A warehouse management system receives products from suppliers while customers simultaneously purchase products.

Develop a multithreaded Producer–Consumer application that:

a) Implements:

- One producer thread.
- One consumer thread.
- A shared buffer of size **5**.
- Synchronization using **mutexes and semaphores**. **(10 Marks)**

The classic solution uses two counting semaphores — empty_slots (starts at BUFFER_SIZE, counts free slots) and full_slots (starts at 0, counts filled slots) — plus a mutex to protect the buffer indices themselves while an item is being inserted or removed.

b) Display all production and consumption activities, including:

- Product number.
- Buffer status.
- Thread performing the operation. **(4 Marks)**

Every line that is printed already has the product number, the thread that is doing the work ([Producer] or [Consumer]) and the number of items in the buffer compared to the buffers size of 5. This covers all three required pieces of information. Since the consumer is a little slower than the producer 150 ms compared to 100 ms) you should see the number of items in the buffer go up to 5 and the producer stop sometimes (using sem_wait(&empty_slots)) until the consumer gets back up to speed

c) Explain how mutexes and semaphores prevent data inconsistency in the shared buffer. **(2 Marks)**

The two semaphores make sure the buffer does not go over its limit. The empty_slots semaphore stops the producer from adding items when the buffer is full. The full_slots semaphore stops the consumer from taking items when the buffer's empty. This way neither thread ever tries to access a slot that's not in the right state. The mutex then protects the shared variables that track where items are added and removed and how many are, in the buffer. It does this during the time when an item is actually being added or taken out. This ensures the producer and consumer never try to change those variables at the same time and mess them up.

d) Include screenshots showing:

- Successful compilation.
- Execution.
- Producer and consumer operations.
- Buffer updates. **(4 Marks)**

```c

#define BUFFER_SIZE 5
#define ITEMS_TO_PRODUCE 10

int buffer[BUFFER_SIZE];
int in = 0, out = 0;
int count = 0;

sem_t empty_slots;
sem_t full_slots;
pthread_mutex_t buffer_lock;

void *producer(void *arg) {
    for (int i = 1; i <= ITEMS_TO_PRODUCE; i++) {
        sem_wait(&empty_slots);
        pthread_mutex_lock(&buffer_lock);

        buffer[in] = i;
        in = (in + 1) % BUFFER_SIZE;
        count++;
        printf("[Producer] produced product #%d | buffer count: %d/%d\n",
               i, count, BUFFER_SIZE);

        pthread_mutex_unlock(&buffer_lock);
        sem_post(&full_slots);
        usleep(100000);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 1; i <= ITEMS_TO_PRODUCE; i++) {
        sem_wait(&full_slots);
        pthread_mutex_lock(&buffer_lock);

        int product = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
        count--;
        printf("[Consumer] consumed product #%d | buffer count: %d/%d\n",
               product, count, BUFFER_SIZE);

        pthread_mutex_unlock(&buffer_lock);
        sem_post(&empty_slots);
        usleep(150000);
    }
    return NULL;
}

int main(void) {
    pthread_t prod, cons;

    sem_init(&empty_slots, 0, BUFFER_SIZE);
    sem_init(&full_slots, 0, 0);
    pthread_mutex_init(&buffer_lock, NULL);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    sem_destroy(&empty_slots);
    sem_destroy(&full_slots);
    pthread_mutex_destroy(&buffer_lock);

    printf("All products have been produced and consumed.\n");
    return 0;
```

```
drip@fedora:~/Downloads/Week6 termpaper$ gcc question3.c -o question3
drip@fedora:~/Downloads/Week6 termpaper$ ./question3
[Producer] produced product #1 | buffer count: 1/5
[Consumer] consumed product #1 | buffer count: 0/5
[Producer] produced product #2 | buffer count: 1/5
[Consumer] consumed product #2 | buffer count: 0/5
[Producer] produced product #3 | buffer count: 1/5
[Consumer] consumed product #3 | buffer count: 0/5
[Producer] produced product #4 | buffer count: 1/5
[Producer] produced product #5 | buffer count: 2/5
[Consumer] consumed product #4 | buffer count: 1/5
[Producer] produced product #6 | buffer count: 2/5
[Consumer] consumed product #5 | buffer count: 1/5
[Producer] produced product #7 | buffer count: 2/5
[Producer] produced product #8 | buffer count: 3/5
[Consumer] consumed product #6 | buffer count: 2/5
[Producer] produced product #9 | buffer count: 3/5
[Consumer] consumed product #7 | buffer count: 2/5
[Producer] produced product #10 | buffer count: 3/5
[Consumer] consumed product #8 | buffer count: 2/5
[Consumer] consumed product #9 | buffer count: 1/5
[Consumer] consumed product #10 | buffer count: 0/5
All products have been produced and consumed.
drip@fedora:~/Downloads/Week6 termpaper$
```

Question Four (20 Marks)

Thread Synchronization Using Condition Variables

A university examination system should generate the final class report **only after all departments have submitted their marks**.

Develop a multithreaded program where:

- Four worker threads represent different academic departments.
- Each department processes marks independently.
- A reporting thread waits until all departments complete processing.
- Synchronization is achieved using **condition variables (pthread_cond_wait() and pthread_cond_signal())**.

Your program should:

a) Display the processing progress of each department. **(6 Marks)**

b) Ensure that the reporting thread prints the final report **only after** all departments finish processing. **(6 Marks)**

c) Explain why condition variables are preferred over busy waiting in this scenario. **(2 Marks)**

A busy-wait alternative would have the reporting thread sit in a tight while (departments_done < NUM_DEPARTMENTS) {} loop, repeatedly checking the counter — burning CPU cycles the whole time and even competing with the department threads for CPU time, which can slow them down further. pthread_cond_wait() instead puts the reporting thread to sleep, consuming no CPU at all, and

atomically releases the mutex while it sleeps so the department threads can safely update departments_done; the OS wakes the reporting thread only when pthread_cond_signal() is actually called, making this both far more efficient and simpler to reason about.

d) Include screenshots showing:

- Source code.
- Compilation.
- Program execution.

Final synchronized report. **(6 Marks)**

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_DEPARTMENTS 4

const char *department_names[NUM_DEPARTMENTS] = {
    "Computer Science", "Mathematics", "Physics", "Business Studies"
};

int departments_done = 0;
pthread_mutex_t lock;
pthread_cond_t all_done_cond;

void *processDepartment(void *arg) {
    int id = *(int *) arg;

    printf("[%s] Processing marks...\n", department_names[id]);
    sleep(1 + id);   /* Simulating processing time */
    printf("[%s] Marks processing complete.\n", department_names[id]);

    pthread_mutex_lock(&lock);
    departments_done++;
    if (departments_done == NUM_DEPARTMENTS) {
        pthread_cond_signal(&all_done_cond);   /* Signal reporting thread */
    }
    pthread_mutex_unlock(&lock);

    return NULL;
}

void *reportingThread(void *arg) {
    pthread_mutex_lock(&lock);
    while (departments_done < NUM_DEPARTMENTS) {
        pthread_cond_wait(&all_done_cond, &lock);   /* Sleep until signaled */
    }
    pthread_mutex_unlock(&lock);

    printf("\n    FINAL CLASS REPORT    \n");
```

```c
    printf("\n=== FINAL CLASS REPORT ===\n");
    printf("All %d departments have submitted their marks.\n", NUM_DEPARTMENTS);
    printf("Final report generated successfully.\n");
    return NULL;
}

int main(void) {
    pthread_t dept_threads[NUM_DEPARTMENTS], report_thread;
    int ids[NUM_DEPARTMENTS];

    pthread_mutex_init(&lock, NULL);
    pthread_cond_init(&all_done_cond, NULL);

    pthread_create(&report_thread, NULL, reportingThread, NULL);

    for (int i = 0; i < NUM_DEPARTMENTS; i++) {
        ids[i] = i;
        pthread_create(&dept_threads[i], NULL, processDepartment, &ids[i]);
    }

    for (int i = 0; i < NUM_DEPARTMENTS; i++) {
        pthread_join(dept_threads[i], NULL);
    }
    pthread_join(report_thread, NULL);

    pthread_mutex_destroy(&lock);
    pthread_cond_destroy(&all_done_cond);
    return 0;
}
```

```
drip@fedora:~/Downloads/Week6 termpaper$ gcc question4.c -o question4
drip@fedora:~/Downloads/Week6 termpaper$ ./question4
[Computer Science] Processing marks...
[Mathematics] Processing marks...
[Physics] Processing marks...
[Business Studies] Processing marks...
[Computer Science] Marks processing complete.
[Mathematics] Marks processing complete.
[Physics] Marks processing complete.
[Business Studies] Marks processing complete.

=== FINAL CLASS REPORT ===
All 4 departments have submitted their marks.
Final report generated successfully.
drip@fedora:~/Downloads/Week6 termpaper$
```

Question Five (20 Marks)
Multithreaded Inventory Management System (Mini Project)
A supermarket wishes to automate stock updates for different product categories.

Develop a complete multithreaded inventory management system that:
a) Creates **four worker threads**, each responsible for updating one of the following categories:

- Electronics
- Groceries
- Clothing
- Stationery

Each thread should:

- Update inventory quantities.
- Display the category being processed.
- Display the updated stock level.
- Simulate processing delays using sleep(). **(8 Marks)**

b) Protect all shared inventory data using a mutex and ensure that no race conditions occur. **(4 Marks)**
c) After all worker threads complete, the main thread should display:

- Final inventory summary.
- Total number of stock updates performed.
- Confirmation that inventory synchronization completed successfully. **(4 Marks)**

d) Include screenshots showing:

- Source code.
- Successful compilation.
- Thread execution.
- Final inventory report. **(4 Marks)**

Submission Requirements
Each student must submit a single report containing:

1. **Cover Page** with unit code, unit title, student name, registration number, and date.
2. **Source Code** for all five questions.
3. **Screenshots** of:
    - Source code.
    - Successful compilation.
    - Program execution.
    - Terminal outputs.

C question5.c > ...

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_CATEGORIES 4
#define UPDATES_PER_CATEGORY 5

const char *categories[NUM_CATEGORIES] = {
    "Electronics", "Groceries", "Clothing", "Stationery"
};

int stock[NUM_CATEGORIES] = {50, 200, 80, 150};
int total_updates = 0;
pthread_mutex_t inventory_lock;

void *updateCategory(void *arg) {
    int id = *(int *) arg;

    for (int i = 0; i < UPDATES_PER_CATEGORY; i++) {
        sleep(1);   /* Simulate delay */

        pthread_mutex_lock(&inventory_lock);
        stock[id] += 10;
        total_updates++;
        printf("[%s] Update #%d -> New stock level: %d units\n",
               categories[id], i + 1, stock[id]);
        pthread_mutex_unlock(&inventory_lock);
    }
    return NULL;
}

int main(void) {
    pthread_t threads[NUM_CATEGORIES];
    int ids[NUM_CATEGORIES];

    pthread_mutex_init(&inventory_lock, NULL);

    for (int i = 0; i < NUM_CATEGORIES; i++) {
        ids[i] = i;
        pthread_create(&threads[i], NULL, updateCategory, &ids[i]);
    }
    for (int i = 0; i < NUM_CATEGORIES; i++) {
        pthread_join(threads[i], NULL);
    }

    pthread_mutex_destroy(&inventory_lock);

    printf("\nFINAL INVENTORY SUMMARY\n");
    for (int i = 0; i < NUM_CATEGORIES; i++) {
        printf("%-12s : %d units\n", categories[i], stock[i]);
    }
    printf("Total stock updates performed: %d\n", total_updates);
    printf("Inventory synchronization completed successfully.\n");
    return 0;
}
```

```
drip@fedora:~/Downloads/Week6 termpaper$ gcc question5.c -o question5
drip@fedora:~/Downloads/Week6 termpaper$ ./question5
 [Groceries] Update #1 -> New stock level: 210 units
 [Stationery] Update #1 -> New stock level: 160 units
 [Clothing] Update #1 -> New stock level: 90 units
 [Electronics] Update #1 -> New stock level: 60 units
 [Groceries] Update #2 -> New stock level: 220 units
 [Stationery] Update #2 -> New stock level: 170 units
 [Clothing] Update #2 -> New stock level: 100 units
 [Electronics] Update #2 -> New stock level: 70 units
 [Groceries] Update #3 -> New stock level: 230 units
 [Stationery] Update #3 -> New stock level: 180 units
 [Clothing] Update #3 -> New stock level: 110 units
 [Electronics] Update #3 -> New stock level: 80 units
 [Groceries] Update #4 -> New stock level: 240 units
 [Clothing] Update #4 -> New stock level: 120 units
 [Stationery] Update #4 -> New stock level: 190 units
 [Electronics] Update #4 -> New stock level: 90 units
 [Stationery] Update #5 -> New stock level: 200 units
 [Clothing] Update #5 -> New stock level: 130 units
 [Groceries] Update #5 -> New stock level: 250 units
 [Electronics] Update #5 -> New stock level: 100 units

 FINAL INVENTORY SUMMARY
 Electronics  : 100 units
 Groceries    : 250 units
 Clothing     : 130 units
 Stationery   : 200 units
 Total stock updates performed: 20
 Inventory synchronization completed successfully.
drip@fedora:~/Downloads/Week6 termpaper$
```

Brief explanation (150–250 words) for each question describing:

- The problem addressed.
- The synchronization mechanisms used.
- Challenges encountered.
- Lessons learned.

Question One: Thread Creation and Management

Problem Addressed: Creating and managing concurrent worker tasks while making sure that thread parameter passing does not cause data corruption because of race conditions on global variables.

Synchronization Mechanisms Used: No explicit locking mechanism (mutex) was needed for reading input parameters because each worker thread was given a pointer to its private TaskArg structure. Process-level synchronization was done using pthread_join() which made the main execution thread wait until all five worker threads were done before printing the completion message.

Challenges Encountered: Making sure that task numbers were sent as heap or stack structure pointers and not as a pointer to the loop counter i directly which caused multiple threads to read the task numbers.

Lessons Learned: Giving thread- context through isolated parameters ensures correctness without the cost of mutex locking. Also pthread_join() is important to stop the thread from ending before the worker threads are done.

Question Two: Race Conditions and Mutex Synchronization

Problem Addressed: Dealing with access to a shared customer bank account balance where multiple ATM terminals do deposit operations at the same time.

Synchronization Mechanisms Used: A POSIX exclusion lock (pthread_mutex_t) that was set up with pthread_mutex_init(). The part of the code that updated balance += DEPOSIT_AMOUNT was protected using pthread_mutex_lock() and pthread_mutex_unlock().

Challenges Encountered: Showing the race condition in the version without synchronization needed a high number of iterations (10,000 deposits per thread) so that the CPU thread interleavings happened often enough to show lost updates.

Lessons Learned: Operations that look like lines in C (like +=) are made up of multiple assembly instructions (load, increment, store). Without exclusion concurrent access can cause lost updates and results that are not predictable. Mutexes make sure data stays correct across shared memory.

Question Three: Producer–Consumer Problem

Problem Addressed: Making sure a producer writing data and a consumer reading data can work together using a shared buffer with a fixed size (5) without the buffer getting full or empty.

Synchronization Mechanisms Used: Two POSIX counting semaphores (empty_slots and sem_t full_slots) to track if there is space or data in the buffer and a POSIX mutex (pthread_mutex_t buffer_lock) to make sure updates to the buffer pointers (in, out) and element counts happen one at a time.

Challenges Encountered: Putting sem_wait() calls inside the mutex lock of outside which caused possible deadlocks where a thread held the mutex while waiting on a semaphore.

Lessons Learned: Counting semaphores are good for managing resources and tracking capacity while mutexes make sure one thread can access data at a time. It is very important to get the order of semaphore wait operations compared to mutex locks to avoid deadlocks.

Question Four: Thread Synchronization Using Condition Variables

Problem Addressed: Making sure a reporting thread waits for four independent academic department threads to finish processing their marks before creating a report.

Synchronization Mechanisms Used: POSIX condition variables (pthread_cond_t all_done_cond) with a POSIX mutex (pthread_mutex_t lock). The reporting thread used pthread_cond_wait(). The last department thread used pthread_cond_signal().

Challenges Encountered: Avoiding wakeups (false wakeups) by putting pthread_cond_wait() inside a while (departments_done < NUM_DEPARTMENTS) loop instead of just an if condition.

Lessons Learned: Condition variables are much better than waiting loops because they let the thread sleep. This uses no CPU time. Helps worker threads run better.

Question Five: Multithreaded Inventory Management System

Problem Addressed: Managing real-time stock inventory updates for four product categories at the time while keeping an accurate global transaction log counter.

Synchronization Mechanisms Used: A POSIX mutex (pthread_mutex_t inventory_lock) to protect access to each category's stock array slots (stock[id]) and the global shared update counter (total_updates).

Challenges Encountered: Adding network or processing delays with sleep() while making sure the lock was only held during shared memory operations and released before sleeping so other threads were not blocked.

Lessons Learned: Making sections as small, as possible helps increase thread concurrency and system performance. After all worker threads are joined with pthread_join() the main thread can. Show final results without needing any locks.

References

1. Class notes and lab manuals given by the lecturer
2. W. R. Stevens and S. A. Rago, Advanced Programming in the UNIX Environment, 3rd ed. Boston, MA: Addison-Wesley Professional, 2013.
3. Generative AI: I used Claude for some refences, understandig some of the required content and also it helped me fix some disturbing bugs in my source codes.